

Lab Work Section 1

Overview

This lab work continues the exploration of pointers in C, reinforcing concepts of declaration, dereferencing, pointer arithmetic, and multi-level pointers with new challenges.

Part 1: Basic Pointer Declaration & Dereferencing (15 pts)

Problem 1: Long Integer Pointer Basics (5 pts)

- **Function Name:** `longPointerOps`
- **Return Type:** void
- **Parameters:** None
- **Description:**
 1. Declare a long integer variable `bigNum` initialized to 100000.
 2. Declare a pointer `ptr` and assign the address of `bigNum` to it.
 3. Print the value of `bigNum`.
 4. Print the value of `bigNum` using `ptr`.
 5. Print the address of `bigNum`.
 6. Print the address stored in `ptr`.

Problem 2: Double Pointer Basics (5 pts)

- **Function Name:** `doublePointerOps`
- **Return Type:** void
- **Parameters:** None
- **Description:**
 1. Declare a double variable `gravity` initialized to 9.81.
 2. Declare a double pointer `p_gravity` pointing to `gravity`.
 3. Print the value of `gravity` directly and via `p_gravity`.
 4. Print the memory address of `gravity`.

Problem 3: Character Pointer - Change Case (5 pts)

- **Function Name:** `charChangeCase`
- **Return Type:** void
- **Parameters:** None
- **Description:**
 1. Declare a char variable `ch` initialized to 'a'.
 2. Declare a char pointer `p_ch` pointing to `ch`.
 3. Change the value of `ch` to 'A' (uppercase) using **only** the pointer `p_ch`.
 4. Print the new value of `ch`.

Part 2: Updating Variables Using Pointers (15 pts)

Problem 4: Modifying Float (5 pts)

- **Function Name:** `modifyBalance`
- **Return Type:** `void`
- **Parameters:** `None`
- **Description:**
 1. Declare a float `balance = 100.50`.
 2. Declare a pointer `p_bal` pointing to `balance`.
 3. Update `balance` to 250.75 using **only** `p_bal`.
 4. Print `balance` to verify.

Problem 5: Modifying Temperature (5 pts)

- **Function Name:** `updateTemperature`
- **Return Type:** `void`
- **Parameters:** `None`
- **Description:**
 1. Declare a double `temp = 36.5`.
 2. Create a pointer to it.
 3. Change the value to 37.2 using the pointer.
 4. Print the new value.

Problem 6: Clamp Value (5 pts)

- **Function Name:** `clampValue`
- **Return Type:** `void`
- **Parameters:** `int *p_val, int min, int max`
- **Description:** Check if the value pointed to by `p_val` is less than `min` or greater than `max`.
 - If less than `min`, set it to `min`.
 - If greater than `max`, set it to `max`.
- **Example:**
 - Input: `val = 150, min = 0, max = 100`
 - Call: `clampValue(&val, 0, 100)`
 - Result: `val` becomes 100

Part 3: Pointer Arithmetic on Arrays (20 pts)

Problem 7: Double Array Traversal (7 pts)

- **Function Name:** `printDoubleArrayPtr`
- **Return Type:** `void`
- **Parameters:** `double *arr, int size`
- **Description:** Print all elements of the double array using pointer arithmetic (incrementing the pointer). **Do not use array indexing `arr[i]`.**

Problem 8: Average of Array (7 pts)

- **Function Name:** `avgArrayPtr`
- **Return Type:** `float`
- **Parameters:** `int *arr, int size`

- **Description:** Calculate and return the average of all elements in the integer array using pointer arithmetic.

Problem 9: Find Maximum (6 pts)

- **Function Name:** `findMaxPtr`
- **Return Type:** `int`
- **Parameters:** `int *arr, int size`
- **Description:** Find and return the maximum value in the array using pointer arithmetic.

Part 4: Functions and Pointers (25 pts)

Problem 10: Swap Floats (8 pts)

- **Function Name:** `swapFloats`
- **Return Type:** `void`
- **Parameters:** `float *a, float *b`
- **Description:** Swap the values of two floats pointed to by `a` and `b`.

Problem 11: Negate (8 pts)

- **Function Name:** `negate`
- **Return Type:** `void`
- **Parameters:** `int *n`
- **Description:** Multiply the value at address `n` by -1.

Problem 12: Rectangle Stats (9 pts)

- **Function Name:** `rect_stats`
- **Return Type:** `void`
- **Parameters:** `int width, int height, int *area, int *perimeter`
- **Description:** Calculate area ($w * h$) and perimeter ($2 * (w + h)$), storing the results in the locations pointed to by `area` and `perimeter`.

Part 5: Double Pointers (25 pts)

Problem 13: Double Pointer to Float (8 pts)

- **Function Name:** `demonstrateDoublePtrFloat`
- **Return Type:** `void`
- **Parameters:** None
- **Description:**
 1. Create a float `f = 3.14`.
 2. Create a pointer `ptr` pointing to `f`.
 3. Create a double pointer `pptr` pointing to `ptr`.
 4. Print `f` using `ptr` and `pptr`.

Problem 14: Modify Char via Double Pointer (8 pts)

- **Function Name:** `modifyCharViaDoublePtr`

- **Return Type:** void
- **Parameters:** `char **pp`
- **Description:** Change the value of the character pointed to by the pointer pointed to by `pp` to 'Z'.

Problem 15: Pointer Reassignment (9 pts)

- **Function Name:** `pointToMax`
- **Return Type:** void
- **Parameters:** `int **pp, int *a, int *b`
- **Description:** Update the pointer pointed to by `pp` so that it points to the larger of the two values pointed to by `a` and `b`.

Submission Guidelines

1. Submit a single `.c` file containing all implementations.
2. Include comments explaining your logic.
3. Test your code with a `main` function that calls all these functions.

Grading

- Correct implementation of Part 1: 15 points
- Correct implementation of Part 2: 15 points
- Correct implementation of Part 3: 20 points
- Correct implementation of Part 4: 25 points
- Correct implementation of Part 5: 25 points
- Code style and documentation: Additional consideration

Total: 100 points